

Relatório técnico e funcional - FinTrackApp

Data: 25/05/2026

Projeto analisado: FinTrackAppExpo / FinTrackApp

Tipo: aplicativo mobile de controle financeiro pessoal com backend próprio para autenticação, dados financeiros persistidos e gestão manual de contas.

1. Visão geral

O FinTrackApp é um aplicativo de finanças pessoais desenvolvido com **Expo + React Native** no frontend e **Node.js + Express + MongoDB/Mongoose** no backend. A experiência principal é mobile, em orientação retrato, com tema escuro, navegação por abas inferiores e fluxos separados para autenticação, início, saldo, análises, transações e perfil.

A versão atual já possui autenticação real via API própria, persistência local de sessão com AsyncStorage, rotas protegidas por JWT e dados financeiros persistidos no MongoDB. Transações, contas e categorias passaram a ser isoladas por usuário, evitando que dados de um usuário apareçam para outro.

O app também passou a trabalhar com **contas manuais vinculadas**. Como não existe integração direta com bancos, o usuário informa manualmente a conta na tela de nova transação. Cada transação atualiza o saldo da conta escolhida, e a tela de saldo soma todas as contas para exibir o saldo total.

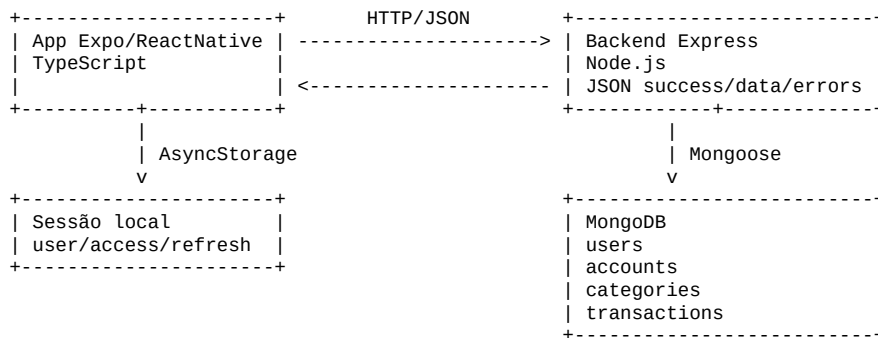
2. Tecnologias usadas

Camada	Tecnologia	Uso no projeto
Aplicativo mobile	Expo ~54.0.33	Runtime, build e execução do app React Native.
UI mobile	React Native 0.81.5	Construção das telas, listas, formulários, navegação e estilos.
Linguagem do app	TypeScript ~5.9.2	Tipagem do frontend, navegação, contratos de API e estruturas de dados.
Framework de UI	React 19.1.0	Componentização, hooks, estado local e contexto global de autenticação.
Navegação	React Navigation 7	Bottom tabs, native stack navigation e recarregamento por foco.
Ícones	@expo/vector-icons / react-native-vector-icons	Ícones Ionicons e MaterialCommunityIcons.
Persistência local	AsyncStorage	Armazenamento local de usuário, access token e refresh token.
Backend	Node.js + Express 4	API REST, middlewares, rotas e regras de negócio financeiras.
Banco de dados	MongoDB + Mongoose 7	Persistência de usuários, transações, contas e categorias.
Autenticação	JWT + bcryptjs	Access token, refresh token, hash de senha e sessão multi-dispositivo.
Segurança API	express-rate-limit + express-validator	Rate limit de login e validação de payload.
Configuração	dotenv	Variáveis de ambiente do backend.
Qualidade	ESLint / eslint-config-expo	Lint do projeto Expo.

3. Estrutura do projeto

```
FinTrackApp/
+- App.tsx                # Bootstrap do app, providers e navegação raiz
+- app.json               # Configuração Expo
+- package.json           # Dependências e scripts do frontend
+- app/
| +- components/common/   # Componentes reutilizáveis de UI
| +- context/AuthContext.tsx # Estado global de autenticação
| +- navigation/          # Stacks, abas e tipos de navegação
| +- screens/             # Telas do aplicativo
| +- theme/index.ts       # Cores, espaçamentos, tipografia e sombras
| +- utils/               # API, storage e mockData/tipos
+- backend/
| +- package.json         # Dependências e scripts do backend
| +- src/
| | +- config/database.js  # Conexão MongoDB
| | +- middleware/auth.js  # Proteção por Bearer token
| | +- models/User.js      # Usuários/autenticação
| | +- models/Account.js   # Contas financeiras manuais
| | +- models/Category.js  # Categorias de entrada/saída
| | +- models/Transaction.js # Transações por usuário e conta
| | +- routes/auth.js      # Endpoints de autenticação
| | +- routes/data.js      # Endpoints financeiros persistidos
| | +- utils/jwt.js        # Geração e verificação JWT
| | +- server.js           # Entrada do servidor Express
+- assets/images/         # Ícones, splash e imagens Expo
```

4. Arquitetura geral



O app decide a rota inicial em `RootNavigator`: se existe usuário autenticado, mostra as abas principais; se não existe, mostra o fluxo de login/cadastro. O contexto de autenticação tenta restaurar a sessão local, valida o usuário com `/auth/me` e, se necessário, renova tokens com `/auth/refresh`.

As telas financeiras consomem a API autenticada. O backend usa `req.user._id` em todas as consultas financeiras, garantindo isolamento por usuário. A tela de nova transação permite selecionar ou criar uma conta manual; o backend registra a transação, vincula à conta e atualiza o saldo da conta.

5. Frontend

5.1 Framework e linguagem

O frontend usa **Expo**, **React Native** e **TypeScript**. O arquivo `App.tsx` configura:

- `GestureHandlerRootView` para gestos.
- `SafeAreaProvider` para áreas seguras em iOS/Android.
- `NavigationContainer` para navegação.
- `AuthProvider` para estado global de autenticação.
- `StatusBar` com estilo claro.

- SplashScreen controlada manualmente durante a inicialização.

5.2 Design system

O tema fica em `app/theme/index.ts` e define uma identidade visual escura:

Elemento	Definição
Fundo principal	#0A0F1E
Cards	#1E2A3A, #243040
Acento	Verde #4FFFB0
Feedback financeiro	Verde para entradas/economia, vermelho para saídas, azul/âmbar/roxo para categorias e contas.
Tipografia	Escalas de h1, h2, body, caption, label.
Espaçamento	Escala de 4 a 32 px.
Bordas	Radius de 8 a 24 px e full pill.

5.3 Componentes reutilizáveis

Componente	Função
AmountText	Exibe valores monetários com cor/estilo conforme entrada ou saída.
Avatar	Mostra iniciais ou avatar do usuário.
Badge	Indicadores pequenos de status/categoria.
Card	Contêiner visual com tema do app.
Header	Cabeçalho de tela com título e ações.
ScreenWrapper	Wrapper padrão com safe area, scroll e fundo.
SectionTitle	Título de seção.
TransactionItem	Linha de transação com ícone, categoria, data e valor.

5.4 Navegação e atualização de dados

```
RootNavigator
+- AuthNavigator
| +- Login
| +- Register
+- MainTabNavigator
  +- Home
  | +- HomeMain
  | +- Categories
  | +- CategoryDetail
  +- Balance
  +- Analytics
  +- Transactions
  | +- TransactionsMain
  | +- AddTransaction
  +- Profile
  +- ProfileMain
  +- EditProfile
  +- Security
  +- AppSettings
  +- Help
```

As telas financeiras foram ajustadas para recarregar dados ao ganharem foco usando padrões do React Navigation. Isso evita a necessidade de sair e entrar novamente no app para visualizar transações, saldos ou categorias recém-criados.

6. Backend

6.1 Framework, linguagem e banco

O backend usa **JavaScript em Node.js**, com **Express** para API REST e **Mongoose** para acesso ao MongoDB. A entrada é `backend/src/server.js`, que aplica CORS, JSON body parser, registra rotas e só inicia o servidor após conectar ao banco.

6.2 Endpoints de autenticação

Método	Rota	Protegida	Finalidade
GET	/	Não	Health check simples do backend.
POST	/auth/register	Não	Cria usuário local com nome, e-mail e senha.
POST	/auth/login	Não	Autentica e retorna usuário, access token e refresh token.
POST	/auth/social-login	Não	Cria/recupera conta social por provider/providerId.
POST	/auth/refresh	Não	Rotaciona refresh token e emite novo access token.
POST	/auth/logout	Sim	Remove o refresh token do dispositivo atual.
POST	/auth/logout-all	Sim	Remove todos os refresh tokens do usuário.
GET	/auth/me	Sim	Retorna o usuário autenticado.

6.3 Endpoints financeiros

Método	Rota	Protegida	Persistência atual	Finalidade
GET	/data/transactions	Sim	MongoDB	Lista transações do usuário autenticado.
POST	/data/transactions	Sim	MongoDB	Cria transação, vincula conta e atualiza saldo.
GET	/data/accounts	Sim	MongoDB	Lista contas manuais do usuário.
GET	/data/categories	Sim	MongoDB	Lista categorias de saída com estatísticas.
GET	/data/categories?type=income	Sim	MongoDB	Lista categorias de entrada: Transferência bancária e Pix.
GET	/data/categories?type=expense	Sim	MongoDB	Lista categorias de saída.
GET	/data/summary	Sim	MongoDB	Retorna entradas, saídas, saldo e contagens.

6.4 Modelos de dados

Modelo	Arquivo	Responsabilidade
User	backend/src/models/User.js	Usuário, login local/social, refresh tokens e dados de segurança.
Account	backend/src/models/Account.js	Conta financeira manual, saldo, ícone, banco/tipo e usuário dono.
Category	backend/src/models/Category.js	Categoria de entrada ou saída, orçamento, cor, ícone e usuário dono.
Transaction	backend/src/models/Transaction.js	Lançamento financeiro vinculado a usuário, conta, categoria, data e tipo.

6.5 Isolamento por usuário

Todas as rotas financeiras usam o usuário autenticado como filtro. A consulta de transações, contas e categorias sempre inclui o identificador do usuário. Com isso, os dados são persistidos no MongoDB e ficam isolados por conta.

```

Usuário autenticado
-> req.user._id
-> Account.find({ user: req.user._id })
-> Category.find({ user: req.user._id })
-> Transaction.find({ user: req.user._id })

```

6.6 Regras financeiras atuais

Regra	Comportamento
Conta padrão	Se o usuário ainda não tiver conta, o backend cria uma "Carteira principal".
Conta manual	A tela de transação permite criar uma nova conta informando nome e tipo/banco.
Saldo da conta	Ao criar uma transação, o backend soma ou subtrai o valor no saldo da conta vinculada.
Saldo total	A tela de saldo soma o saldo de todas as contas vinculadas.
Entrada	Valor positivo. Categorias permitidas: Transferência bancária e Pix.
Saída	Valor negativo. Categorias existentes são mantidas.
Categorias por usuário	Categorias padrão são criadas automaticamente para cada usuário.

7. API no frontend

O arquivo `app/utils/api.ts` centraliza chamadas HTTP. A URL atual está fixa como:

```
export const API_BASE_URL = "http://192.168.1.70:4000";
```

Também existe uma função `getDebugHost`, mas ela não está sendo usada na constante final. Para desenvolvimento em múltiplos computadores, emuladores e redes, o ideal é trocar essa URL fixa por variável de ambiente ou lógica baseada no host do Expo.

Chamadas principais:

- `loginWithEmail`
- `registerWithEmail`
- `logout`

- refreshAccessToken
- getCurrentUser
- getTransactions
- createTransaction
- getAccounts
- getCategories, agora com filtro opcional income ou expense.
- authFetch com renovação automática do token em caso de 401.

8. Estado atual dos dados

Área	Estado atual
Usuários	Persistidos no MongoDB.
Senhas	Hash bcrypt no backend.
Sessão mobile	Persistida no AsyncStorage.
Tokens	Access token e refresh token com rotação.
Transações	Persistidas no MongoDB e isoladas por usuário.
Contas	Persistidas no MongoDB, criadas manualmente via tela de transação e isoladas por usuário.
Categorias	Persistidas no MongoDB, criadas por usuário e separadas por tipo de entrada/saída.
Resumo financeiro	Calculado a partir de transações e contas persistidas.
Saldo total	Soma dos saldos das contas vinculadas.

9. Funcionalidades recentes adicionadas

Funcionalidade	Descrição	Impacto
Persistência financeira	Transações, contas e categorias agora usam MongoDB.	Dados permanecem após reiniciar servidor/app.
Isolamento por usuário	Todos os dados financeiros são filtrados por <code>req.user._id</code> .	Privacidade e separação correta entre contas.
Recarregamento por foco	Telas financeiras recarregam ao voltar para elas.	Dados novos aparecem sem logout/login.
Contas vinculadas manuais	Usuário escolhe ou cria conta na tela de nova transação.	App controla saldo por conta sem conexão bancária.
Saldo total por contas	Tela de saldo soma os saldos das contas vinculadas.	O saldo total reflete a soma real dos lançamentos por conta.
Categorias por tipo	Entrada usa Pix/Transferência; saída mantém categorias de gasto.	Formulário fica mais coerente e reduz erro de classificação.

10. Telas e mockups

Os mockups abaixo são wireframes textuais baseados nos componentes e textos encontrados no código. Eles representam estrutura, hierarquia e ações esperadas de cada tela.

10.1 Login

Arquivo: app/screens/Auth/LoginScreen.tsx

Objetivo: autenticar usuário existente e permitir navegação para cadastro.

```
+-----+
|           [wallet]           |
|           FinTrack           |
| Controle total do seu dinheiro |
+-----+
| Bem-vindo de volta          |
| Entre na sua conta para continuar |
|                               |
| E-mail                      |
| [ seu@email.com             ] |
| Senha                       |
| [ *****                   ] |
|                               |
|                               | Esqueceu a senha? |
|                               |
| [ Entrar                    ] |
|                               |
|                               | ou continue com |
| [ Google ]                  | [ Facebook ] |
|                               |
| Não tem conta? Cadastre-se  |
+-----+
```

10.2 Cadastro

Arquivo: app/screens/Auth/RegisterScreen.tsx

Objetivo: criar conta local com nome, e-mail, senha e confirmação.

```
+-----+
|           [wallet]           |
|           FinTrack           |
| Cadastre-se e comece a controlar |
| seu dinheiro                 |
+-----+
| Crie sua conta              |
| Preencha os dados abaixo    |
| Nome completo              |
| [ Seu nome                  ] |
| E-mail                     |
| [ seu@email.com             ] |
| Senha                      |
| [ *****                   ] |
| Força da senha [barra]      |
| Confirmar senha            |
| [ Repita a senha            ] |
| [ Cadastrar                 ] |
| Já tem uma conta? Entrar    |
+-----+
```

10.3 Início

Arquivo: app/screens/Home/HomeScreen.tsx

Objetivo: apresentar visão rápida do saldo, entradas, saídas, atalhos e transações recentes.

```
+-----+
| Bom dia, [Nome]             [avatar] |
+-----+
| Saldo disponível           |
| R$ 12.847,50               |
| +3,2% este mês             |
+-----+
| Entradas                   Saídas      |
| R$ 4.200,00                R$ 1.832,00 |
+-----+
| Atalhos                    |
| [Transações] [Análises] [Categorias] |
| [Saldo]                   |
+-----+
| Recentes                   Ver todas |
| - Pix recebido             +R$ ...  |
| - Mercado Extra           -R$ ...  |
+-----+
```

```

| - Uber                                -R$ ... |
+-----+
| Início | Saldo | Análises | Trans. |

```

10.4 Categorias

Arquivo: app/screens/Categories/CategoriesScreen.tsx

Objetivo: listar categorias de gastos, total do mês e distribuição percentual. Esta tela usa categorias de saída.

```

+-----+
| <- Categorias                        |
+-----+
| TOTAL GASTO EM MAI                  |
| R$ 1.832,00                         |
+-----+
| Mercado      R$ 680   37%   >      |
| [barra de progresso]                |
| Transporte    R$ 410   22%   >      |
| [barra de progresso]                |
| Alimentação   R$ 328   18%   >      |
| Entretenimento R$ 205   11%   >      |
| Saúde         R$ 120    7%    >      |
| Presentes     R$  89    5%    >      |
+-----+

```

10.5 Detalhe da categoria

Arquivo: app/screens/Categories/CategoryDetailScreen.tsx

Objetivo: exibir detalhes de uma categoria selecionada.

```

+-----+
| <- [Categoria]                      |
+-----+
| [ícone]                             |
| Total gasto                          |
| R$ xxx,xx                           |
+-----+
| Transações      [quantidade]         |
| Percentual      [xx%]                |
| Orçamento       [R$ xxx,xx]          |
+-----+
| Progresso do orçamento                |
| [#####.....]                       |
+-----+

```

10.6 Saldo

Arquivo: app/screens/Balance/BalanceScreen.tsx

Objetivo: consolidar saldo total e listar contas vinculadas manuais.

```

+-----+
| Saldo                               |
| Resumo financeiro de [usuário]      |
+-----+
| SALDO TOTAL                         |
| R$ 12.847,50                        |
| +R$1.360 economizados este mês     |
+-----+
| Entradas este mês                   |
| [barra] R$ 4.200 / R$ 4.200         |
| Saídas este mês                     |
| [barra] R$ 1.832 / R$ 4.200         |
+-----+
| Contas vinculadas                   |
| [ícone] Nubank                      |
|      Conta corrente      R$ 8.240   |
| [ícone] Itaú              |
|      Poupança            R$ 3.607   |
| [ícone] Carteira principal          |
|      Saldo geral         R$ 1.000   |
+-----+

```


10.7 Análises

Arquivo: app/screens/Analytics/AnalyticsScreen.tsx

Objetivo: mostrar indicadores e gráficos/visualizações de entradas, saídas e economia. As categorias de maiores gastos usam apenas categorias de saída.

Análises			
[Dia] [Semana] [Mês]			
Entradas	Saídas	Economia	
R\$...	R\$...	R\$...	
Entradas vs. Saídas			
Jan	#### entrada	###	saída
Fev	#### entrada	###	saída
Mar	#### entrada	###	saída
Abr	#####	##	
Mai	####	##	
Maiores gastos			
Mercado		37%	
Transporte		22%	
Alimentação		18%	
Início	Saldo	Análises	Trans.

10.8 Transações

Arquivo: app/screens/Transactions/TransactionsScreen.tsx

Objetivo: listar e buscar transações, além de abrir o cadastro de nova transação. A tela recarrega ao ganhar foco.

Transações		[+]
[Buscar transação...		x]
[Todas] [Entradas] [Saídas]		
[Mercado] [Transporte]		
Hoje		
- Pix recebido		+R\$...
- Mercado Extra		-R\$...
- Uber		-R\$...
Estado vazio:		
Nenhuma transação encontrada		
Início Saldo Análises Trans.		

10.9 Nova transação

Arquivo: app/screens/Transactions/AddTransactionScreen.tsx

Objetivo: cadastrar entrada ou saída financeira, escolhendo conta vinculada e categoria compatível com o tipo.

<- Nova Transação	
Nome da Transação	
[Ex: Compra no Mercado	]
Tipo de Transação	
[Entrada]	[Saída]
Valor	
R\$ [0,00	]
Data	

```

| [ YYYY-MM-DD ] |
+-----+
| Conta vinculada |
| [Nubank R$ 8.240] [Itaú R$ 3.607] |
| [Nova conta manual] |
| Se Nova conta: |
| [ Ex: Nubank ] |
| [ Tipo ou banco: conta corrente ] |
+-----+
| Categoria |
| Se Entrada: [Transferência] [Pix] |
| Se Saída: [Mercado] [Transporte] |
|           [Alimentação] [...] |
+-----+
| Resumo |
| Transação: ... |
| Data: ... |
| Categoria: ... |
| Tipo: Entrada/Saída |
| Conta: ... |
| Valor: ... |
+-----+
| [ Cancelar ] [ Adicionar ] |
+-----+

```

10.10 Perfil

Arquivo: app/screens/Profile/ProfileScreen.tsx

Objetivo: mostrar dados do usuário, estatísticas rápidas, plano e atalhos de configuração.

```

+-----+
| Perfil                                cfg |
+-----+
| [avatar/câmera] |
| Nome do usuário |
| e-mail          |
| Plano Básico    |
+-----+
| 47      6      3 |
| Transações  Categorias  Contas |
+-----+
| Editar perfil > |
| Segurança > |
| Configurações do app > |
| Ajuda > |
+-----+
| [ Sair ] |
+-----+
| Início | Saldo | Análises | Trans. |

```

10.11 Editar perfil

Arquivo: app/screens/Profile/EditProfileScreen.tsx

Estado atual: tela placeholder com cabeçalho e mensagem para implementação.

```

+-----+
| <- Editar Perfil |
+-----+
| Implemente o conteúdo desta tela |
| aqui |
+-----+

```

10.12 Segurança

Arquivo: app/screens/Profile/SecurityScreen.tsx

Estado atual: tela placeholder com cabeçalho e mensagem para implementação.

```

+-----+
| <- Segurança |
+-----+

```

```

| Implemente o conteúdo desta tela |
| aqui                             |
+-----+

```

10.13 Configurações do app

Arquivo: app/screens/Profile/AppSettingScreen.tsx

Estado atual: tela placeholder com cabeçalho e mensagem para implementação.

```

+-----+
| <- Configurações                |
+-----+
| Implemente o conteúdo desta tela |
| aqui                             |
+-----+

```

10.14 Ajuda

Arquivo: app/screens/Profile/HelpScreen.tsx

Estado atual: tela placeholder com cabeçalho e mensagem para implementação.

```

+-----+
| <- Ajuda                        |
+-----+
| Implemente o conteúdo desta tela |
| aqui                             |
+-----+

```

11. Fluxos principais

11.1 Login

```

Usuário abre app
-> AuthContext tenta restaurar tokens do AsyncStorage
-> Se não houver sessão: Login
-> POST /auth/login
-> Backend valida e-mail/senha
-> Backend retorna user + accessToken + refreshToken
-> App salva no AsyncStorage
-> RootNavigator mostra MainTabNavigator

```

11.2 Renovação de sessão

```

App restaura accessToken e refreshToken
-> GET /auth/me
-> Se accessToken estiver inválido/expirado
-> POST /auth/refresh
-> Backend valida refreshToken salvo no usuário
-> Backend remove token antigo e gera token novo
-> App salva novos tokens

```

11.3 Criação de transação com conta manual

```

Usuário abre Transações
-> Toca em +
-> Escolhe Entrada ou Saída
-> App busca categorias por tipo
  -> Entrada: Transferência bancária, Pix
  -> Saída: Mercado, Transporte, Alimentação, Entretenimento, Saúde, Presentes
-> Usuário escolhe conta existente ou cria Nova conta
-> POST /data/transactions com Bearer token
-> Backend resolve/cria conta do usuário
-> Backend grava transação no MongoDB
-> Backend atualiza saldo da conta
-> Telas recarregam ao ganhar foco

```

11.4 Saldo total

```
GET /data/accounts
-> Backend retorna contas do usuário
-> Tela de saldo soma account.balance de todas as contas
-> Campo SALDO TOTAL exibe a soma
-> Lista Contas vinculadas exibe cada conta e seu saldo individual
```

12. Pontos fortes

- Separação clara entre frontend, backend, navegação, tema, contexto e utilitários.
- Autenticação com fluxo robusto para login, cadastro, refresh token e logout.
- Uso de hash bcrypt e remoção de campos sensíveis no JSON do usuário.
- Dados financeiros agora persistidos no MongoDB e isolados por usuário.
- Contas manuais permitem controlar saldos sem integração bancária externa.
- Categorias de entrada e saída foram separadas, reduzindo classificação incorreta.
- Interface mobile consistente, com tema próprio e componentes reutilizáveis.
- Navegação bem estruturada entre autenticação, abas principais e stacks internos.
- Telas financeiras recarregam ao voltar ao foco, melhorando a experiência após cadastro de transações.

13. Lacunas e riscos atuais

Área	Observação	Impacto
URL da API	IP local fixo em API_BASE_URL.	App pode falhar em outro ambiente/rede.
Edição/exclusão	Não há endpoints/telas para editar ou excluir transações e contas.	Correções manuais ainda exigem intervenção no banco.
Saldo inicial	Nova conta começa com saldo zero e recebe saldo por lançamentos.	Para cadastrar saldo já existente, usuário precisa criar uma entrada manual.
Transferências entre contas	Entrada possui categoria "Transferência bancária", mas não existe fluxo de débito/crédito entre duas contas.	Transferências podem inflar entradas se usadas como movimentação interna.
Login social	Backend aceita provider/providerId, mas não valida token com Google/Facebook/X.	Risco de autenticação social falsa se exposto em produção.
Telas de perfil	Editar perfil, segurança, configurações e ajuda ainda são placeholders.	Funcionalidades de conta ficam incompletas.
README	Ainda é majoritariamente o README padrão do Expo.	Falta documentação de instalação real do frontend/backend.
Testes	Não foram identificados testes automatizados.	Maior risco de regressões em autenticação e finanças.

14. Recomendações técnicas

1. Substituir o IP fixo da API por variável de ambiente Expo ou configuração por ambiente.
2. Criar endpoints e telas para editar/excluir transações.
3. Criar fluxo dedicado para saldo inicial de conta, separado de entradas recorrentes.
4. Implementar transferência entre contas como operação própria: debita uma conta e credita outra.
5. Criar tela de gestão de contas para cadastrar, renomear, arquivar e ajustar saldo.

6. Validar login social no backend usando tokens reais dos provedores.
7. Implementar as telas placeholder do perfil.
8. Criar testes para autenticação, refresh token, rotas protegidas, transações, contas e categorias por tipo.
9. Atualizar o README com setup real de frontend, backend, MongoDB e variáveis de ambiente.
10. Adicionar tratamento visual padronizado para loading/erro nas telas financeiras.

15. Scripts úteis

Frontend:

```
npm install
npm run start
npm run android
npm run ios
npm run web
npm run lint
```

Backend:

```
cd backend
npm install
npm run dev
npm start
```

Variáveis esperadas no backend:

```
MONGODB_URI=mongodb+srv://...
JWT_SECRET=...
JWT_REFRESH_SECRET=...
PORT=4000
```

16. Conclusão

O FinTrackApp evoluiu de um protótipo com dados financeiros em mock/memória para uma base funcional com persistência real no MongoDB, isolamento por usuário, contas manuais vinculadas e categorização mais coerente entre entradas e saídas. O fluxo financeiro atual já permite ao usuário registrar transações, controlar saldos por conta e visualizar o saldo total consolidado.

O próximo avanço natural é amadurecer a gestão financeira: edição/exclusão de lançamentos, cadastro direto de contas, saldo inicial, transferências entre contas e testes automatizados. Com esses pontos, o projeto fica mais próximo de um app de finanças pessoais utilizável no dia a dia.